ЗВІТ ДО ПРОГРАМНОГО ПРОЄКТУ №4

Виконала: Ткаченко Анастасія, група К-25, варіант 13

Тема: Структура даних із m = 2 цілих полів. Частки операцій від усіх операцій: поле 0: read 10%, write 10%; поле 1: read 50%, write 10%; string 20%.

Схема захисту даних: використано 2 м'ютекси (std::mutex): m0 захищає поле 0, m1 захищає поле 1. Операції read/write беруть лише м'ютекс свого поля. Операція string читає обидва поля, тому бере обидва м'ютекси одночасно, завжди в порядку m0 -> m1, і отримує узгоджений знімок. Лише string тримає два замки, а порядок захоплення фіксований, тому взаємоблокування неможливе.

Поля незалежні, тож один спільний м'ютекс без потреби блокував би операції над різними полями, а розділення дає паралелізм між ними (на поле 1 припадає 80% операцій, на поле 0 лише 40%). Критичні секції дуже короткі (одне читання або присвоєння int), тому shared_mutex не використано: його накладні витрати перевищили б виграш від паралельних читачів. Кожен м'ютекс разом зі своїм полем вирівняно по 64 байти (alignas(64)), щоб уникнути false sharing між потоками.

Середній час у секундах (середнє з 3 запусків). Release x64, MSVC, C++20, усього 1 200 000 операцій незалежно від кількості потоків. Файл «a» має частоти за умовою, «b» має рівні частоти (по 20%), «c» підібрано вручну: write 0 = 60%, string = 30%, read 0 = 5%, write 1 = 4%, read 1 = 1%.

Файл	1 потік	2 потоки	3 потоки
a	0.0604	0.0743	0.0890
b	0.0604	0.0782	0.0875
c	0.0663	0.0693	0.0752

Висновки: Таблиця лише частково відповідає очікуванням. Схема коректна (без data race та взаємоблокувань), але додаткові потоки не прискорюють роботу, а сповільнюють її: при переході від 1 до 3 потоків час зростає приблизно на 45-50% у файлах «a» та «b». Причина в тому, що критичні секції надзвичайно короткі, тож основний час іде на змагання за м'ютекси та перенесення кеш-рядків між ядрами, а не на корисну роботу. Різниця між файлами «a», «b», «c» мала, оскільки вузьке місце становить вартість самого блокування, а не розподіл операцій. Дещо менше зростання часу у файлі «c» (0.0663 -> 0.0752) можна пояснити тим, що там навантаження зосереджено на одному полі, але це припущення.

Самостійно: Складено демо-код (генерація й зчитування файлів, запуск у 1-3 потоках, усереднення за 3 запусками) і підібрано власний розподіл частот для файлу «c». Розглянуто shared_mutex та mutex для поля 1 і обрано звичайний mutex. Під час перших запусків час був нереально малим (0.0001 с); причину знайдено: потоки jthread не завершувались до взяття t1, і це виправлено окремим блоком. Проведено вимірювання в режимі Release x64.